

Tu fais partie d'une équipe qui développe une application React de gestion des tâches. Le projet est hébergé sur **GitLab** et suit un **workflow professionnel** (branches, MR, conflits, corrections, rollback).

PARTIE 0 – Préparation

1. Configure Git (nom et email).
2. Crée le projet React avec Vite.
3. Initialise Git.

Solution :

```
git config --global user.name "Nom Prenom"
git config --global user.email "email@mail.com"

npm create vite@latest react-task-manager
cd react-task-manager

git init
git status
```

PARTIE 1 – Premier commit & .gitignore

1. Ajoute .gitignore (node_modules).
2. Ajoute tous les fichiers du projet.
3. Fais un commit nommé :
"Initialisation projet React"
4. Supprime un fichier inutile et commit.

Solution :

```
git add .
git commit -m "Initialisation projet React"

git rm src/assets/react.svg
git commit -m "Suppression assets inutiles"

git log --oneline
```

PARTIE 2 – GitLab : remote, push, clone

1. Crée un dépôt vide sur GitLab.
2. Lien le dépôt local.
3. Pousse le projet.
4. Clone le projet dans un autre dossier.

```
git remote add origin https://gitlab.com/user/react-task-manager.git
git branch -M main
git push -u origin main
cd ..
git clone https://gitlab.com/user/react-task-manager.git
```

PARTIE 3 – Développement par branches

1. Crée la branche feature-task-list.
2. Implémente un composant TaskList.jsx.
3. Commit.
4. Liste les branches.
5. Reviens à main.

Solution :

```
git switch -c feature-task-list
git add src/components/TaskList.jsx
git commit -m "Ajout composant TaskList"
git branch
git switch main
```

PARTIE 4 – Merge & conflits

1. Modifie App.jsx sur main.
2. Commit.
3. Reviens sur feature-task-list.
4. Modifie aussi App.jsx.
5. Merge → résoudre conflit.

PARTIE 5 – Annulation & restauration

1. Annule une modification non commitée.
2. Annule un commit en gardant les fichiers.
3. Restaure un fichier supprimé.

Solution :

```
git restore src/App.jsx
git reset --soft HEAD~1
git checkout HEAD~1 -- src/components/TaskList.jsx
```

PARTIE 6 – Historique & inspection

1. Affiche l'historique compact.
2. Affiche le contenu d'un commit.
3. Identifie qui a modifié une ligne.
4. Récupère un commit perdu.

Solution :

```
git log --oneline --graph  
git show <commit_id>  
git blame src/App.jsx  
git reflog
```

PARTIE 7 – Collaboration GitLab (fork & upstream)

1. Fork le projet.
2. Clone ton fork.
3. Ajoute le dépôt original comme upstream.
4. Récupère les mises à jour.

Solution :

```
git remote add upstream https://gitlab.com/original/react-task-  
manager.git  
git fetch upstream  
git pull upstream main
```

PARTIE 8 – Revert & version stable

1. Annule un commit instable.
2. Crée un tag v1.0.0.

Solution :

```
git revert <commit_id>  
git tag v1.0.0  
git push origin v1.0.0
```

PARTIE 9 – Nettoyage & maintenance

1. Sauvegarde temporairement des modifications.
2. Nettoie les fichiers non suivis.

Solution :

git stash

git stash pop

git clean -fd